

Strings - Simple Hinglish Summary

Source: <https://javascript.info/string>

Main baat

JavaScript me text data **strings** me store hota hai. Ek single character ke liye **alag type nahi** hai. Strings ka internal format hamesha **UTF-16** hota hai, page ki encoding se koi lena-dena nahi.

1. Quotes

Strings **single quotes, double quotes ya backticks** me likh sakte hain:

```
javascript
```

```
let single = 'single-quoted';
let double = "double-quoted";

let backticks = `backticks`;
```

Single aur double quotes **essentially same** hain. **Backticks** me kisi bhi **expression** ko `${...}` me daalkar string me embed kar sakte hain:

```
javascript
```

```
function sum(a, b) {
  return a + b;
}

alert(`1 + 2 = ${sum(1, 2)}.`); // 1 + 2 = 3.
```

Backticks ka ek aur fayda: string **kai lines** me likh sakte hain:

```
javascript
```

```
let guestList = `Guests:
* John
* Pete
* Mary
`;

alert(guestList); // kai lines ki list
```

Single ya double quotes me aisa karo to **error** aata hai:

javascript

```
let guestList = "Guests: // Error: Unexpected token ILLEGAL  
  * John";
```

Single/double quotes language ke shuruaati zamane ke hain jab multiline strings ki zarurat nahi socchi gayi thi. Backticks baad me aaye, isliye zyada versatile hain.

(Backticks se pehle "template function" bhi laga sakte hain: `func`string``). Ise **tagged templates** kehte hain, ye bahut kam dikhta hai.)

2. Special characters

Single/double quotes se bhi multiline string bana sakte hain **newline character** (`\n`) se:

javascript

```
let guestList = "Guests:\n * John\n * Pete\n * Mary";  
  
alert(guestList); // upar jaisi hi multiline list
```

Ye dono strings barabar hain:

javascript

```
let str1 = "Hello\nWorld"; // "newline symbol" se do lines  
  
// normal newline aur backticks se do lines  
let str2 = `Hello  
World`;  
  
alert(str1 == str2); // true
```

Aur special characters:

Character	Matlab
<code>\n</code>	Nayi line
<code>\r</code>	Windows text files me <code>\r\n</code> line break hota hai. Baaki OS me sirf <code>\n</code> . Historical reasons se
<code>\'</code> , <code>\"</code> , <code>\`</code>	Quotes
<code>\\</code>	Backslash
<code>\t</code>	Tab
<code>\b</code> , <code>\f</code> , <code>\v</code>	Backspace, Form Feed, Vertical Tab (purane zamane ke, ab use nahi hote, bhool sakte ho)

Saare special characters **backslash** (`\`) (escape character) se shuru hote hain. Asli backslash dikhana ho to use **double** karo:

```
javascript
alert( `The backslash: \\` ); // The backslash: \
```

Escaped quotes (`\'`, `\"`, `\``) tab kaam aate hain jab same quote wali string ke andar quote daalna ho:

```
javascript
alert( 'I\'m the Walrus!' ); // I'm the Walrus!
```

Sirf wahi quote escape karna padta hai jo bahar wale quote ke same ho. Behtar tarika: double quotes ya backticks use karo:

```
javascript
alert( "I'm the Walrus!" ); // I'm the Walrus!
```

(Unicode ke liye `\u...` notation bhi hai, ye kam use hota hai.)

3. String length

`length` property string ki lambai deti hai:

javascript

```
alert( `My\n`.length ); // 3
```

`\n` ek **single** special character hai, isliye length `3` hai.

`length` **property** hai, **function** nahi. `str.length()` nahi, sirf `str.length` likhna hai.

4. Characters access karna

Position `pos` par character lene ke liye `[pos]` ya `str.at(pos)`. Pehla character position `0` par hota hai:

javascript

```
let str = `Hello`;  
  
// pehla character  
alert( str[0] );    // H  
alert( str.at(0) ); // H  
  
// aakhri character  
alert( str[str.length - 1] ); // o  
alert( str.at(-1) );          // o
```

`.at(pos)` ka fayda: **negative position** allow karta hai. Negative `pos` ho to **end se gina** jata hai. `.at(-1)` aakhri character, `.at(-2)` uske pehle wala, aise.

Square brackets negative index par hamesha `undefined` dete hain:

javascript

```
let str = `Hello`;  
  
alert( str[-2] );    // undefined  
alert( str.at(-2) ); // l
```

`for...of` se characters par loop chalta hai:

javascript

```
for (let char of "Hello") {  
  alert(char); // H, e, l, l, o  
}
```

5. Strings immutable hain (badal nahi sakte)

JS me strings **badli nahi ja sakti**. Ek character bhi badalna namumkin hai:

```
javascript

let str = 'Hi';

str[0] = 'h'; // error
alert( str[0] ); // kaam nahi karta
```

Aam workaround: poori nayi string banao aur use purani ki jagah assign karo:

```
javascript

let str = 'Hi';

str = 'h' + str[1]; // string replace ki

alert( str ); // hi
```

6. Case badalna

`toLowerCase()` aur `toUpperCase()`:

```
javascript

alert( 'Interface'.toUpperCase() ); // INTERFACE
alert( 'Interface'.toLowerCase() ); // interface
```

Sirf ek character lowercase karna ho:

```
javascript

alert( 'Interface'[0].toLowerCase() ); // 'i'
```

7. Substring dhundhna

`str.indexOf(substr, pos)`

`str` me `substr` ko `pos` position se dhundhta hai. Milne par **position** return karta hai, nahi mile to `-1`.

```
javascript
```

```
let str = 'Widget with id';

alert( str.indexOf('Widget') ); // 0 (shuru me mila)
alert( str.indexOf('widget') ); // -1 (nahi mila, search case-sensitive hai)

alert( str.indexOf("id") ); // 1
```

Optional dusra parameter search ko kisi position se shuru karta hai:

```
javascript

let str = 'Widget with id';

alert( str.indexOf('id', 2) ) // 12
```

Saare occurrences chahiye to `indexOf` ko loop me chalao, har baar pichhle match ke baad ki position se:

```
javascript

let str = 'As sly as a fox, as strong as an ox';

let target = 'as';

let pos = 0;
while (true) {
  let foundPos = str.indexOf(target, pos);
  if (foundPos == -1) break;

  alert( `Found at ${foundPos}` );
  pos = foundPos + 1;
}
```

`str.lastIndexOf(substr, position)`: end se shuru ki taraf dhundhta hai (occurrences ulte order me).

`indexOf` ko `if` me galat use

```
javascript
```

```
let str = "Widget with id";

if (str.indexOf("Widget")) {
    alert("We found it"); // kaam nahi karta!
}
```

Kyunki `indexOf` yahan `0` return karta hai (shuru me mila), aur `if` `0` ko `false` maanta hai. **Sahi:** `-1` se compare karo:

javascript

```
if (str.indexOf("Widget") !== -1) {
    alert("We found it"); // ab chalega
}
```

`includes`, `startsWith`, `endsWith`

`str.includes(substr, pos)` modern method hai jo `true/false` deta hai. Jab position nahi chahiye, sirf match check karna ho:

javascript

```
alert( "Widget with id".includes("Widget") ); // true
alert( "Hello".includes("Bye") ); // false

alert( "Widget".includes("id") ); // true
alert( "Widget".includes("id", 3) ); // false (position 3 se "id" nahi)
```

`startsWith` aur `endsWith` jaisa naam waisa kaam:

javascript

```
alert( "Widget".startsWith("Wid") ); // true
alert( "Widget".endsWith("get") ); // true
```

8. Substring nikalna

Teen methods hain: `slice`, `substring`, `substr`.

`str.slice(start [, end])`: `start` se `end` tak (`end` شامل nahi):

javascript

```
let str = "stringify";
alert( str.slice(0, 5) ); // 'strin'
alert( str.slice(0, 1) ); // 's'
```

Dusra argument na ho to end tak:

```
javascript

alert( str.slice(2) ); // 'ringify'
```

Negative values end se gine jate hain:

```
javascript

alert( str.slice(-4, -1) ); // 'gif'
```

`str.substring(start [, end])`: `start` aur `end` ke **beech** ka hissa. `slice` jaisa hi, lekin `start > end` allow karta hai (dono swap kar deta hai):

```
javascript

let str = "stringify";

alert( str.substring(2, 6) ); // "ring"
alert( str.substring(6, 2) ); // "ring"

alert( str.slice(2, 6) ); // "ring"
alert( str.slice(6, 2) ); // "" (khali string)
```

Negative arguments support nahi hote, `@` maane jate hain.

`str.substr(start [, length])`: `start` se `length` characters. End position ki jagah length dete hain:

```
javascript

alert( str.substr(2, 4) ); // 'ring'
alert( str.substr(-4, 2) ); // 'gi'
```

Ye Annex B me hai (browser-only features), isliye **recommended nahi**, lekin practice me sab jagah chalta hai.

Method	Kya select karta hai	Negatives
<code>slice(start, end)</code>	<code>start</code> se <code>end</code> tak (<code>end</code> شامل nahi)	Allowed
<code>substring(start, end)</code>	<code>start</code> aur <code>end</code> ke beech	Negative = <code>0</code>
<code>substr(start, length)</code>	<code>start</code> se <code>length</code> characters	<code>start</code> negative allowed

Kaunsa chunein? Sab kaam kar dete hain, lekin `slice` zyada flexible hai (negatives allowed) aur chhota likha jata hai. **Practice me sirf `slice` yaad rakhna kaafi hai.**

9. Strings compare karna

Strings alphabetical order me character-by-character compare hoti hain, lekin kuch ajeeb baatein hain:

1. **Lowercase hamesha uppercase se bada hota hai:**

```
javascript
alert( 'a' > 'Z' ); // true
```

2. **Diacritical marks wale letters "out of order" hote hain:**

```
javascript
alert( 'Österreich' > 'Zealand' ); // true
```

Kyu? JS me strings **UTF-16** me encode hoti hain, yaani har character ka ek **numeric code** hota hai.

`str.codePointAt(pos)`: position `(pos)` ke character ka code deta hai:

```
javascript
alert( "Z".codePointAt(0) ); // 90
alert( "z".codePointAt(0) ); // 122
alert( "z".codePointAt(0).toString(16) ); // 7a
```

`String.fromCodePoint(code)`: code se character banata hai:

```
javascript
alert( String.fromCodePoint(90) ); // Z
alert( String.fromCodePoint(0x5a) ); // Z
```

Characters (65..220) ki string:

```
javascript

let str = '';

for (let i = 65; i <= 220; i++) {
  str += String.fromCharCode(i);
}

alert( str );
// ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
// ¡¢£¥¦§¨ª«¬®¯°±²³´µ¶·¸¹º»¼½¾¿ÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖ×ØÙÚÛÜ
```

Pehle capital letters, phir kuch special, phir lowercase, aur (0) end ke paas. Ab samajh aata hai ki (a > Z) kyu hai: **characters unke numeric code se compare hote hain**, (a) (97) ka code (Z) (90) se bada hai.

Sahi comparison: (localeCompare)

Sahi comparison algorithm complex hai kyunki alag languages ke alphabets alag hote hain. Modern browsers **ECMA-402** internationalization standard support karte hain.

(str.localeCompare(str2)) language ke rules ke hisaab se:

- **Negative number:** (str) chhota hai (str2) se
- **Positive number:** (str) bada hai (str2) se
- **0:** barabar

```
javascript

alert( 'Österreich'.localeCompare('Zealand') ); // -1
```

(Isme do aur arguments hain jo language aur case sensitivity jaisi settings dete hain.)

Summary

- **3 tarah ke quotes.** Backticks se string kai lines me ho sakti hai aur ({...}) se expressions embed ho sakte hain.
- Special characters use kar sakte hain, jaise line break (\n).
- Character lene ke liye: ([]) ya (at).
- Substring ke liye: (slice) ya (substring).
- Lowercase/uppercase: (toLowerCase/toUpperCase).
- Substring dhundhne ke liye: (indexOf), ya simple checks ke liye (includes/startsWith/endsWith).

- Language ke hisaab se compare karne ke liye: `localeCompare`, warna character codes se compare hoti hain.

Aur helpful methods:

- `str.trim()`: shuru aur end ke spaces hata deta hai
- `str.repeat(n)`: string ko `n` baar repeat karta hai
- ...aur bahut se manual me

Regular expressions ke saath search/replace ke methods alag section me hain. Aur ye jaan lo ki strings **Unicode** par based hain, isliye comparison me issues aate hain.

Practice Tasks (Answers)

1. Pehla character uppercase karo (`ucFirst`): Strings immutable hain, isliye nayi string banate hain:

```
javascript

function ucFirst(str) {
  if (!str) return str;

  return str[0].toUpperCase() + str.slice(1);
}

alert( ucFirst("john") ); // John
```

`if (!str) return str;` isliye zaruri hai ki khali string me `str[0]` `undefined` hota hai aur error aata.

2. Spam check (`checkSpam`): Case-insensitive ke liye pehle lowercase karo:

```
javascript

function checkSpam(str) {
  let lowerStr = str.toLowerCase();

  return lowerStr.includes('viagra') || lowerStr.includes('xxx');
}
```

3. Text chhota karo (`truncate`):

```
javascript
```

```
function truncate(str, maxLength) {
  return (str.length > maxLength) ?
    str.slice(0, maxLength - 1) + '...' : str;
}
```

Ellipsis ek single Unicode character (...) hai (teen dots nahi), isliye (maxLength - 1).

4. Paisa nikalo (extractCurrencyValue):

javascript

```
function extractCurrencyValue(str) {
  return +str.slice(1);
}

alert( extractCurrencyValue('$120') === 120 ); // true
```

Quick Summary

Kaam	Kaise
Variable string me daalna	<code>`Hello \${name}`</code> (backticks)
Multiline string	Backticks ya <code>\n</code>
Lambai	<code>str.length</code> (function nahi)
Character lena	<code>str[0]</code> ya <code>str.at(-1)</code>
Badalna	Nahi ho sakta (immutable), nayi string banao
Case	<code>toUpperCase()</code> , <code>toLowerCase()</code>
Position dhundhna	<code>str.indexOf("x")</code> (nahi mile to <code>-1</code>)
Sirf hai ya nahi	<code>includes</code> , <code>startsWith</code> , <code>endsWith</code>
Substring	<code>str.slice(start, end)</code>
Space hatana	<code>str.trim()</code>
Repeat	<code>str.repeat(3)</code>
Sahi compare	<code>str.localeCompare(str2)</code>
<code>'a' > 'z'</code>	<code>true</code> (character codes se compare)